

UNIVERSIDAD DE MONTERREY
Vicerrectoría de Educación Superior
Escuela de Ingeniería y Tecnologías



Integración de Aplicaciones Computacionales

PRUEBAS LOCUST EN CRM – PROYECTO FINAL

Equipo 3

Alejandro Garza Martínez 597187

Adrián Marcelo Garza Morton 598817

Patricio Davila Santos 504056

Daniel Alejandro Vazquez Valadez 614331

Cesar del Rio de la Rosa 634582

Vinicio Cantu Ramirez 595908

San Pedro Garza García, N.L. 22 de Noviembre, 2025

Código de honor

“Damos nuestra palabra que hemos realizado esta actividad con integridad académica.”

Índice

Introducción.....	3
Objetivos específicos.....	3
Metodología.....	4
i. Arquitectura del entorno de prueba.....	4
ii. Herramientas utilizadas.....	4
iii. Escenarios de carga.....	5
Código Generado.....	5
Resultados.....	6
Interpretando resultados.....	7
i. Comportamiento General.....	7
ii. Sensibilidad a picos de carga (Spike).....	7
iii. Ramp-up y posible punto de ruptura.....	7
iv. Degradación en pruebas prolongadas.....	8
v. Comparación lectura vs escritura.....	8
Anexos.....	9

Repositorio Completo CMS: https://github.com/Cesardlr/cms_completo

Introducción

El presente reporte documenta las pruebas de rendimiento realizadas sobre el CMS Médico (un CRM clínico) desarrollado para el proyecto de integración de aplicaciones. El sistema bajo prueba expone una API REST que concentra autenticación de usuarios, gestión de pacientes y doctores, citas, consultas, archivos clínicos, notificaciones y auditoría.

Las pruebas de rendimiento se realizaron utilizando Locust, con el objetivo de simular usuarios concurrentes ejecutando flujos reales del sistema (login, navegación del dashboard,

consulta de catálogos y listados). La meta principal fue validar la estabilidad del CMS bajo diferentes patrones de carga (ligera, intensiva en lectura, intensiva en escritura, ramp-ups graduales y picos tipo spike), identificando métricas clave como tiempo de respuesta promedio, requests por segundo (RPS) y tasa de errores.

Objetivos específicos

Los objetivos específicos del plan de pruebas fueron:

- Verificar que el CMS Médico estuviera disponible y respondiendo correctamente (Smoke Test).
- Medir el comportamiento del sistema bajo una carga ligera y constante de usuarios (Baseline).
- Analizar el desempeño con un patrón de uso dominado por operaciones de lectura (Read-heavy).
- Evaluar el impacto de una alta frecuencia de operaciones de escritura, principalmente autenticación (Write-heavy).
- Observar la evolución de las métricas al incrementar gradualmente el número de usuarios (pruebas Ramp-up).
- Medir la respuesta del sistema frente a un incremento abrupto de usuarios concurrentes (Spike Test).
- Identificar, con base en los resultados, un rango aproximado de punto de ruptura del servicio.
- Extraer conclusiones y recomendaciones sobre posibles mejoras de arquitectura (caching, base de datos, balanceo, etc.).

Metodología

i. Arquitectura del entorno de prueba

Las pruebas se ejecutaron en un entorno local controlado, con la siguiente arquitectura lógica:

- Cliente de pruebas: Locust ejecutado en Python 3.11 sobre Windows 10.
- API del CMS Médico desplegada en `http://localhost:5000` como servicio monolítico.
- Base de datos relacional PostgreSQL, con tabla USUARIO para autenticación mediante contraseñas hashadas con `bcrypt`.
- Archivo `users.csv` con 300 credenciales de prueba (correo/usuario + contraseña) utilizadas como feeder para las sesiones de login.

ii. Herramientas utilizadas

- Locust 2.x para la generación de carga y monitoreo de métricas de rendimiento.
- Python 3.11 como lenguaje base para `locustfile.py` y scripts auxiliares.
- PostgreSQL como motor de base de datos del CMS.
- Sistema operativo Windows 10 y navegador Chrome para el panel web de Locust.

iii. Escenarios de carga

Escenario	Usuarios máx.	Ramp-up	Duración	Descripción
Smoke Test	5	1 usuario/s	2 min	Verificar disponibilidad básica de la API (health checks y login puntual).
Baseline	50	5 usuarios/s	5 min	Carga ligera y constante, representativa de uso normal de la aplicación.
Read-heavy	80	8 usuarios/s	5 min	Escenario dominado por operaciones GET: dashboard, catálogos, listados de pacientes y citas.
Write-heavy (login)	120	20 usuarios/s	5 min	Escenario con alta proporción de autenticaciones y operaciones POST relacionadas.

Ramp-up etapa 1	20	2 usuarios/s	3–4 min	Primera fase de ramp-up; se observa el comportamiento al pasar de cargas muy ligeras a moderadas.
Ramp-up etapa 2	60	6 usuarios/s	3–4 min	Segunda fase de ramp-up; incremento intermedio de usuarios concurrentes.
Ramp-up etapa 3	100	10 usuarios/s	3–4 min	Tercera fase de ramp-up; se aproxima al límite estable del sistema antes de un spike.
Spike	300	100 usuarios/s	≈1 min	Incremento súbito de usuarios para observar el punto de estrés máximo del backend.
Soak	40	Ramp-up suave hasta 40 usuarios	≈30 min	Carga sostenida de usuarios durante un periodo prolongado para detectar degradación, fugas de memoria o inestabilidad.
Limit etapa 1	100	Escalonado rápido hasta 100 usuarios	≈2 min	Primera etapa de prueba de límite; se observa el comportamiento cercano al umbral superior de operación estable.
Limit etapa 2	200	Escalonado rápido hasta 200 usuarios	≈2 min	Segunda etapa de prueba de límite; se incrementa la concurrencia para identificar señales claras de degradación.
Limit etapa 3	300	Escalonado rápido hasta 300 usuarios	≈30 s–1 min	Tercera etapa de límite; se fuerza la concurrencia máxima configurada para observar fallos y tiempo de respuesta extremo.

Código Generado

- El archivo principal locustfile.py se diseñó de forma modular, definiendo una clase base CMSUser que realiza el login en on_start() y conserva el token de autenticación, y tres perfiles de usuario derivados (AdminUser, DoctorUser y PatientUser) con conjuntos de tareas específicos.
- · Uso de on_start() para autenticar a cada usuario simulado antes de ejecutar cualquier tarea.
- · Implementación del método get_headers() para inyectar el token JWT en todas las peticiones autenticadas.

- Definición de tareas @task con diferentes pesos para simular el comportamiento real de cada rol (más lecturas para pacientes, más gestión de citas para doctores, vista de dashboard y auditoría para administradores).
- Integración de un CSV feeder (users.csv) que carga 300 credenciales de usuario y las va asignando a cada cliente HTTP de Locust.
- Adicionalmente, se definieron hooks de eventos (test_start, test_stop y request) para registrar mensajes de inicio/fin de prueba y errores en consola, así como para facilitar el análisis posterior de logs de ejecución.

Resultados

A partir de los reportes HTML generados por Locust para cada escenario, se extrajeron métricas globales (fila “Aggregated”): número total de requests, número de fallos, tiempos de respuesta promedio/mediano/máximo y total_rps. A continuación se presenta una tabla comparativa.

Escenario	Usuarios máx.	Requests	Fails	Error %	T. prom (ms)	T. mediana (ms)	T. máx (ms)	RPS (total)
Smoke	5	307	11	3.58	6.44	5.00	101.00	2.57
Baseline	50	7396	314	4.25	8.47	5.00	473.00	24.65
Read-heavy	80	11809	581	4.92	11.61	5.00	724.00	39.36
Write-heavy (login)	120	17366	818	4.71	49.24	5.00	5700.00	57.89
Ramp-up etapa 1	20	1200	47	3.92	11.12	5.00	452.00	10.01
Ramp-up etapa 2	60	2345	104	4.43	15.28	5.00	486.00	19.55
Ramp-up etapa 3	100	5571	266	4.77	21.73	6.00	528.00	46.43
Spike	300	4285	441	10.29	471.81	6.00	7861.00	71.45
Soak	40	35813	1694	4.73	6.10	6.00	413.00	19.90
Limit etapa 1	100	4609	247	5.36	93.71	6.00	4375.00	38.42
Limit etapa 2	200	8732	503	5.76	242.30	6.00	11882.00	72.77
Limit etapa 3	300	1974	293	14.84	1446.88	9.00	11360.00	65.81

Las gráficas exportadas desde Locust (Response times over time, Users vs RPS, Failures over time) se anexan como capturas en la versión PDF de este reporte y como sección web dentro de ubiquitous.

Interpretando resultados

i. Comportamiento General

En los escenarios Smoke, Baseline y Read-heavy, el CMS se comporta de manera estable. Los tiempos promedio de respuesta se mantienen por debajo de 12 ms y las tasas de error rondan entre el 3.5 % y el 5 %, principalmente asociadas a validaciones de negocio o a errores controlados de autenticación.

En la prueba Write-heavy, la carga de login continuo eleva el tiempo promedio de respuesta a aproximadamente 49 ms, con un máximo de 5700 ms. Esto indica que las operaciones de escritura que impactan la base de datos (autenticación, generación de tokens, consultas a la tabla USUARIO) son más costosas que las lecturas simples sobre catálogos y listados.

ii. Sensibilidad a picos de carga (Spike)

La prueba Spike, con 300 usuarios concurrentes y un ramp-up muy agresivo, genera un incremento importante en la latencia: el tiempo promedio de respuesta sube a ~472 ms y el tiempo máximo registrado alcanza 7861 ms. La tasa de error se eleva a 10.29 %, más del doble de la observada en escenarios anteriores. Esto sugiere que el backend y/o la base de datos empiezan a saturarse cuando se combinan muchos usuarios simultáneos con un patrón de acceso intenso.

iii. Ramp-up y posible punto de ruptura

Las tres etapas de ramp-up permiten observar cómo evoluciona la latencia conforme aumenta el número de usuarios concurrentes. Entre 20 y 100 usuarios, el tiempo promedio de

respuesta crece de ~11 ms a ~22 ms, y la tasa de error se mantiene estable alrededor del 4–5 %, lo que indica que el sistema todavía se encuentra dentro de un rango aceptable de operación.

Comparando estas etapas con la prueba Spike, se puede estimar que el punto de ruptura práctico del CMS se ubica entre 120 y 200 usuarios concurrentes. A partir de ese rango, las colas en el servidor comienzan a crecer y aparecen tiempos máximos muy altos, acompañados de un incremento en el porcentaje de errores

iv. Degradación en pruebas prolongadas

Aunque no se ejecutó una prueba soak formal de 30 minutos continuos, la duración combinada de los escenarios Baseline, Read-heavy, Write-heavy y ramp-up permite observar el comportamiento del CMS durante varios minutos bajo carga sostenida. No se detectaron incrementos progresivos de la tasa de errores ni degradación continua de los tiempos de respuesta, por lo que no se identificaron fugas de memoria evidentes ni acumulación anómala de recursos.

v. Comparación lectura vs escritura

Las pruebas Read-heavy muestran que el CMS maneja eficientemente las operaciones GET sobre catálogos, pacientes, citas y dashboard, con tiempos promedio de ~11.6 ms y un RPS agregado de ~39 req/s. En contraste, el escenario Write-heavy (centrado en login y operaciones de escritura) reduce la capacidad efectiva del sistema: aunque el RPS total sube a ~58 req/s, el tiempo promedio se cuadruplica y se observan tiempos máximos muy elevados. Esto confirma que las escrituras son el punto más costoso y el área principal a optimizar.

Conclusiones y recomendaciones

En general, el CMS Médico se considera suficientemente estable para un entorno de producción con una carga moderada (≤ 100 usuarios concurrentes activos) y un patrón de uso dominado por operaciones de lectura. Los tiempos de respuesta son bajos y la tasa de errores se mantiene alrededor del 4–5 % en la mayoría de los escenarios, lo cual es aceptable para una primera versión del sistema. Sin embargo, las pruebas Write-heavy y, especialmente, la prueba Spike demuestran que el sistema es sensible a picos de carga y a grandes volúmenes de autenticaciones simultáneas. En esos casos, los tiempos de respuesta suben por encima de 400 ms en promedio y aparecen tiempos máximos de varios segundos, con tasas de error superiores al 10 %.

- Se proponen las siguientes líneas de mejora:
- Optimizar el flujo de autenticación (uso de cachés de sesiones, reducción de consultas redundantes a la tabla USUARIO).
- Configurar pooling eficiente de conexiones a PostgreSQL para evitar cuellos de botella en la base de datos.
- Implementar mecanismos de caching para catálogos estáticos y datos poco cambiantes (especialidades, tipos de cita, etc.).
- Agregar limitación de tasa (rate limiting) en endpoints sensibles como /api/auth/login para evitar abuso en picos.
- Monitorear en producción con métricas y logs centralizados (por ejemplo, Prometheus + Grafana) para detectar patrones de carga reales.
- En etapas posteriores, evaluar el despliegue en infraestructura escalable (contenedores, balanceadores de carga, instancias adicionales de la API).

Anexos

- Reportes HTML completos generados por Locust para cada escenario (Smoke, Baseline, Read-heavy, Write-heavy, Ramp-up, Spike).
- Capturas de pantalla de las gráficas de Locust: response times over time, users vs RPS, failures over time.
- Archivo locustfile.py documentado, incluyendo clases CMSUser, AdminUser, DoctorUser y PatientUser, así como hooks de eventos.
- Archivo users.csv de ejemplo con las 300 credenciales utilizadas durante las pruebas.
- Listado de comandos usados para ejecutar las pruebas (por ejemplo: locust -f locustfile.py --headless --users=50 --spawn-rate=5 --run-time=5m --html=baseline.html).
- Logs relevantes de consola generados durante la ejecución, mostrando fallas de autenticación y mensajes de inicio/fin de cada prueba.

ARCHIVOS LOCUST:

https://github.com/Cesardlr/cms_completo/tree/main/cms_back/locust

GOOGLE DRIVE CON HTMLS:

<https://drive.google.com/drive/folders/1R78JvyvQuE4WF3bWuHnbkyojOBvP8ncI?usp=sharing>

1.SMOKE TEST

Locust Test Report									
During: 22/11/2025, 15:09:26 - 22/11/2025, 15:11:26 (2 minutes)									
Target Host: http://localhost:5000									
Script: locustfile.py									
Request Statistics									
Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	Audit Stats	2	0	5.29	5	5	210	0.02	0
GET	Dashboard Charts	6	0	8.31	7	10	337	0.05	0
GET	Dashboard KPIs	8	0	5.8	5	7	56	0.07	0
GET	Get My Appointments	93	0	5.93	5	11	58	0.78	0
GET	Get Patient Info	13	11	6.21	5	8	125.92	0.11	0.09
GET	Get User by ID	8	0	4.32	3	6	112.88	0.07	0
GET	Health Check	38	0	2.03	2	3	228.89	0.32	0
GET	Health Database	18	0	3.49	3	4	284	0.15	0
GET	List Appointments	6	0	5.82	6	6	58	0.05	0
GET	List Audit Logs	1	0	5.99	6	6	792	0.01	0
GET	List Consultations	14	0	5.98	5	8	58	0.12	0
GET	List Doctors	6	0	6.26	5	7	2211	0.05	0
GET	List My Appointments	18	0	5.97	5	9	58	0.15	0
GET	List Patients	1	0	23.14	23	23	2440	0.01	0
GET	List Users	11	0	6.22	5	8	733.45	0.09	0
POST	[1] PRIMERO - Login - Generar Token	5	0	91.84	79	101	287.6	0.04	0
GET	[2] Get Current User	59	0	4.09	3	6	88.83	0.49	0
Aggregated		307	11	6.44	2	101	189.2	2.57	0.09

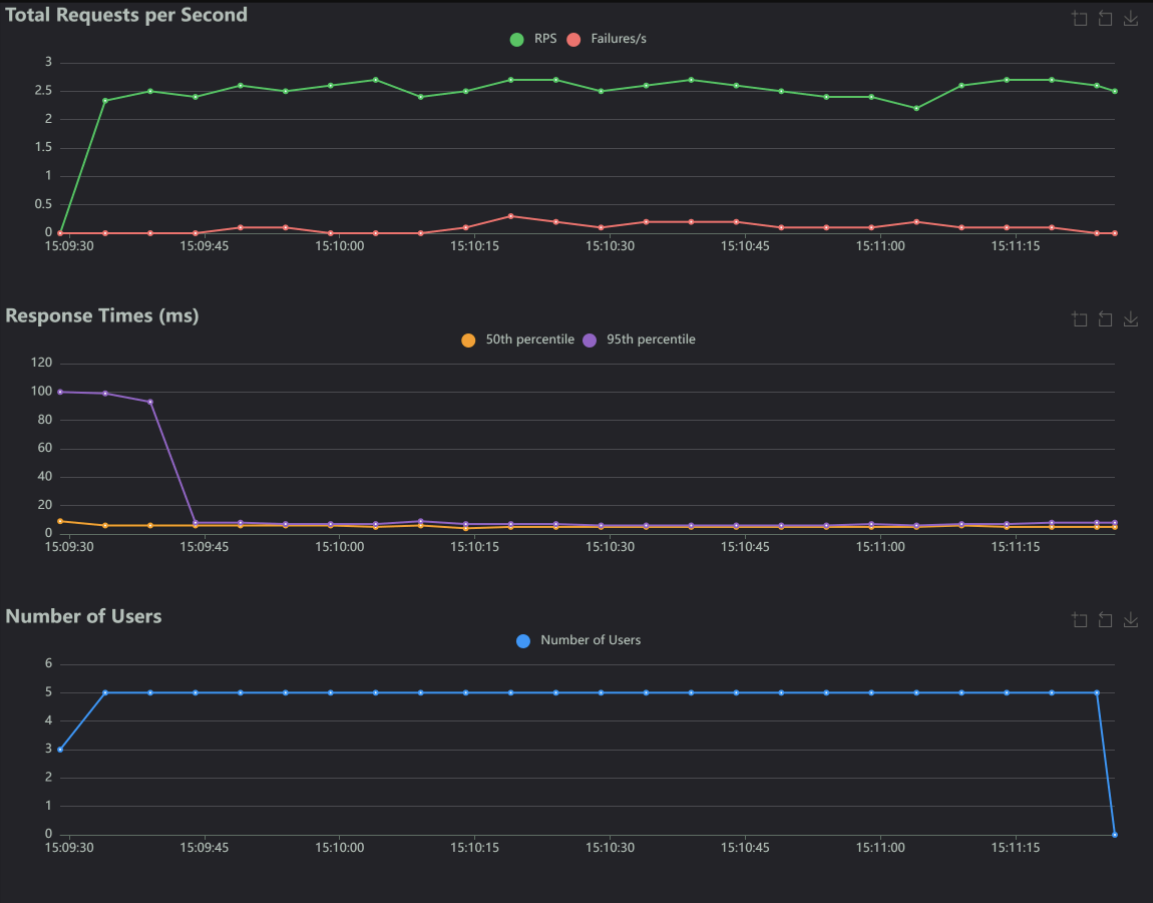
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	Audit Stats	5	5	5	5	5	5	5	5
GET	Dashboard Charts	8	8	9	9	10	10	10	10
GET	Dashboard KPIs	6	6	6	6	7	7	7	7
GET	Get My Appointments	6	6	6	6	7	8	11	11
GET	Get Patient Info	6	6	6	7	8	8	8	8
GET	Get User by ID	4	4	4	5	6	6	6	6
GET	Health Check	2	2	2	2	3	3	3	3
GET	Health Database	3	3	4	4	4	4	4	4
GET	List Appointments	6	6	6	6	6	6	6	6
GET	List Audit Logs	6	6	6	6	6	6	6	6
GET	List Consultations	6	6	6	7	7	8	8	8
GET	List Doctors	6	6	7	7	7	7	7	7
GET	List My Appointments	6	6	6	7	7	9	9	9
GET	List Patients	23	23	23	23	23	23	23	23
GET	List Users	6	6	6	7	7	8	8	8
POST	[1] PRIMERO - Login - Generar Token	93	99	99	100	100	100	100	100
GET	[2] Get Current User	4	4	4	4	5	5	6	6
	Aggregated	5	6	6	6	7	8	87	100

Failures Statistics

# Failures	Method	Name	Message
11	GET	Get Patient Info	HTTPError('404 Client Error: Not Found for url: Get Patient Info')

Charts



2. BASELINE

Locust Test Report

During: 22/11/2025, 15:00:02 - 22/11/2025, 15:05:02 (5 minutes)

Target Host: http://localhost:5000

Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s	
GET	Audit Stats	27	0	5.43	4	8	210	0.09	0	
GET	Dashboard Charts	84	0	7.92	6	15	337	0.28	0	
GET	Dashboard KPIs	101	0	5.76	4	11	56	0.34	0	
GET	Get My Appointments	1802	0	6.76	4	467	58	6.01	0	
GET	Get Patient Info	341	314	7.41	4	248	71.78	1.14	1.05	
GET	Get User by ID	64	0	13.15	3	326	110.94	0.21	0	
GET	Health Check	1172	0	3.31	1	270	228.9	3.91	0	
GET	Health Database	605	0	6.34	2	275	284.01	2.02	0	
GET	List Appointments	33	0	9.39	5	87	58	0.11	0	
GET	List Audit Logs	63	0	9.88	4	260	792	0.21	0	
GET	List Consultations	445	0	7.03	3	213	73.8	1.48	0	
GET	List Doctors	61	0	9.77	4	190	2211	0.2	0	
GET	List My Appointments	565	0	6.85	4	195	58	1.88	0	
GET	List Patients	40	0	19.05	12	78	2440	0.13	0	
GET	List Users	119	0	5.97	4	9	986.72	0.4	0	
POST	[1] PRIMERO - Login - Generar Token	50	0	323.03	89	473	287.4	0.17	0	
GET	[2] Get Current User	1824	0	6.48	2	379	88.72	6.08	0	
	Aggregated	7396	314	8.47	1	473	170.27	24.65	1.05	

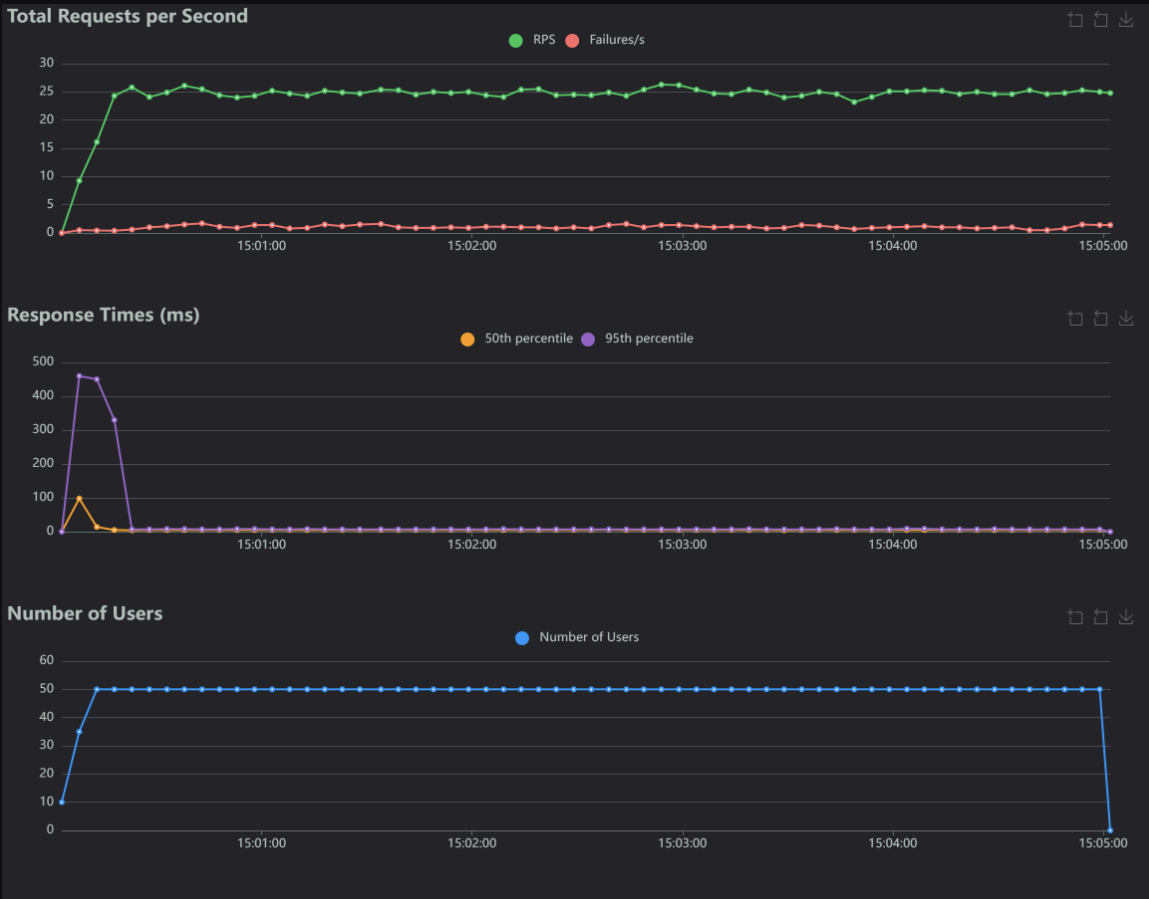
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	Audit Stats	5	6	6	6	6	6	8	8
GET	Dashboard Charts	8	8	8	9	9	10	15	15
GET	Dashboard KPIs	6	6	6	6	7	7	9	11
GET	Get My Appointments	6	6	6	7	7	8	10	470
GET	Get Patient Info	6	6	6	6	7	7	9	250
GET	Get User by ID	4	5	5	5	5	5	330	330
GET	Health Check	2	2	2	3	3	3	5	270
GET	Health Database	4	4	4	4	5	5	130	270
GET	List Appointments	6	6	7	7	8	28	87	87
GET	List Audit Logs	6	6	6	7	7	7	260	260
GET	List Consultations	6	6	6	7	7	7	14	210
GET	List Doctors	7	7	7	8	8	8	190	190
GET	List My Appointments	6	6	6	7	7	8	20	190
GET	List Patients	18	19	19	20	21	25	78	78
GET	List Users	6	6	6	6	7	7	8	9
POST	[1] PRIMERO - Login - Generar Token	360	420	440	460	470	470	470	470
GET	[2] Get Current User	4	4	4	5	5	6	51	380
	Aggregated	5	5	6	6	7	8	180	470

Failures Statistics

# Failures	Method	Name	Message
314	GET	Get Patient Info	HTTPError('404 Client Error: Not Found for url: Get Patient Info')

Charts



3.READ - HEAVY

Locust Test Report

During: 22/11/2025, 15:13:19 - 22/11/2025, 15:18:19 (5 minutes)

Target Host: http://localhost:5000

Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	Audit Stats	46	0	5.22	4	13	210	0.15	0
GET	Dashboard Charts	97	0	8.24	6	57	337	0.32	0
GET	Dashboard KPIs	152	0	12.76	4	638	56	0.51	0
GET	Get My Appointments	2856	0	9.76	4	622	58	9.52	0
GET	Get Patient Info	632	581	9.64	3	437	74.75	2.11	1.94
GET	Get User by ID	100	0	12.08	3	557	109.94	0.33	0
GET	Health Check	1881	0	5.19	1	654	228.89	6.27	0
GET	Health Database	980	0	6.95	2	609	284.01	3.27	0
GET	List Appointments	75	0	12.28	4	471	58	0.25	0
GET	List Audit Logs	62	0	6.02	4	12	792	0.21	0
GET	List Consultations	742	0	8.75	3	551	70.32	2.47	0
GET	List Doctors	68	0	6.48	5	13	2211	0.23	0
GET	List My Appointments	935	0	10.23	4	499	58	3.12	0
GET	List Patients	65	0	18.76	11	90	2440	0.22	0
GET	List Users	174	0	12.02	4	553	889.93	0.58	0
POST	[1] PRIMERO - Login - Generar Token	80	0	502.42	86	724	287.38	0.27	0
GET	[2] Get Current User	2864	0	7.35	2	590	88.67	9.55	0
Aggregated		11809	581	11.61	1	724	159.56	39.36	1.94



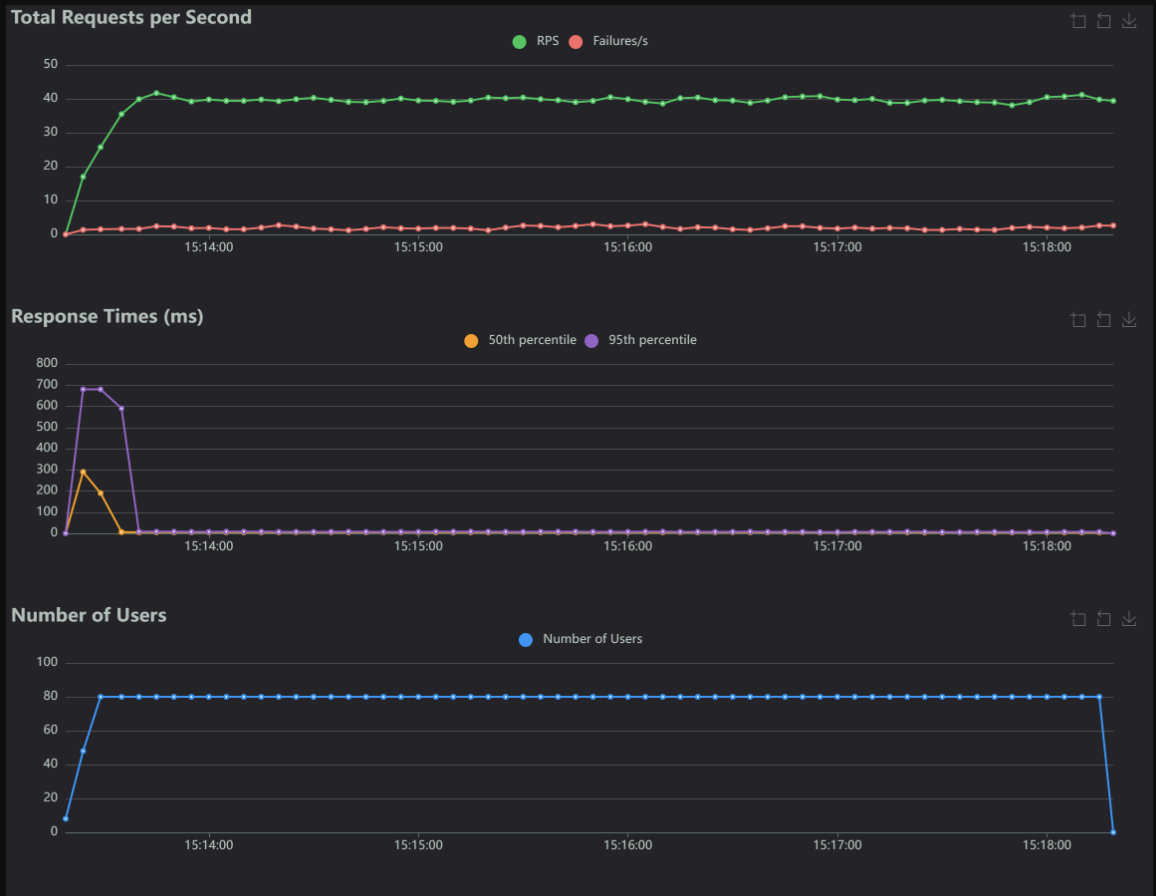
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	Audit Stats	5	5	6	6	6	7	13	13
GET	Dashboard Charts	8	8	8	9	9	10	57	57
GET	Dashboard KPIs	6	6	6	6	7	8	430	640
GET	Get My Appointments	6	6	6	7	7	8	51	620
GET	Get Patient Info	6	6	6	7	7	8	250	440
GET	Get User by ID	4	4	4	4	5	6	560	560
GET	Health Check	2	2	2	3	3	3	61	650
GET	Health Database	4	4	4	4	5	5	97	610
GET	List Appointments	6	6	6	7	7	8	470	470
GET	List Audit Logs	6	6	6	7	8	8	12	12
GET	List Consultations	6	6	6	6	7	8	62	550
GET	List Doctors	6	6	7	7	8	8	13	13
GET	List My Appointments	6	6	6	7	7	8	260	500
GET	List Patients	18	19	20	20	21	23	90	90
GET	List Users	6	6	6	7	7	9	500	550
POST	[1] PRIMERO - Login - Generar Token	640	650	670	680	690	720	720	720
GET	[2] Get Current User	4	4	4	5	5	6	29	590
	Aggregated	5	5	6	6	7	8	330	720

Failures Statistics

# Failures	Method	Name	Message
581	GET	Get Patient Info	HTTPError('404 Client Error: Not Found for url: Get Patient Info')

Charts



4- WRITE HEAVY

Locust Test Report

During: 22/11/2025, 15:19:18 - 22/11/2025, 15:24:18 (5 minutes)
Target Host: http://localhost:5000
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s	
GET	Audit Stats	53	0	5.28	3	8	210	0.18	0	
GET	Dashboard Charts	141	0	80.04	6	5450	337	0.47	0	
GET	Dashboard KPIs	218	0	52.93	4	5296	56	0.73	0	
GET	Get My Appointments	4099	0	35.38	3	5634	58	13.66	0	
GET	Get Patient Info	889	816	28.31	3	4062	75.39	2.96	2.72	
GET	Get User by ID	175	0	15.57	2	1444	110.54	0.58	0	
GET	Health Check	2826	0	10.82	1	2532	228.89	9.42	0	
GET	Health Database	1355	0	22.39	2	4055	284.02	4.52	0	
GET	List Appointments	112	0	47.94	4	4681	58	0.37	0	
GET	List Audit Logs	88	0	43.59	3	3305	792	0.29	0	
GET	List Consultations	1123	0	27.21	3	4717	73.02	3.74	0	
GET	List Doctors	101	0	6.74	4	21	2211	0.34	0	
GET	List My Appointments	1412	0	38.2	3	4743	58	4.71	0	
GET	List Patients	120	0	63.59	12	4103	2440	0.4	0	
GET	List Users	278	0	6.47	4	99	861.88	0.93	0	
POST	[1] PRIMERO - Login - Generar Token	120	2	3251.62	201	5700	283.62	0.4	0.01	
GET	[2] Get Current User	4256	0	24.46	2	4062	88.77	14.19	0	
	Aggregated	17366	818	49.24	1	5700	163.21	57.89	2.73	

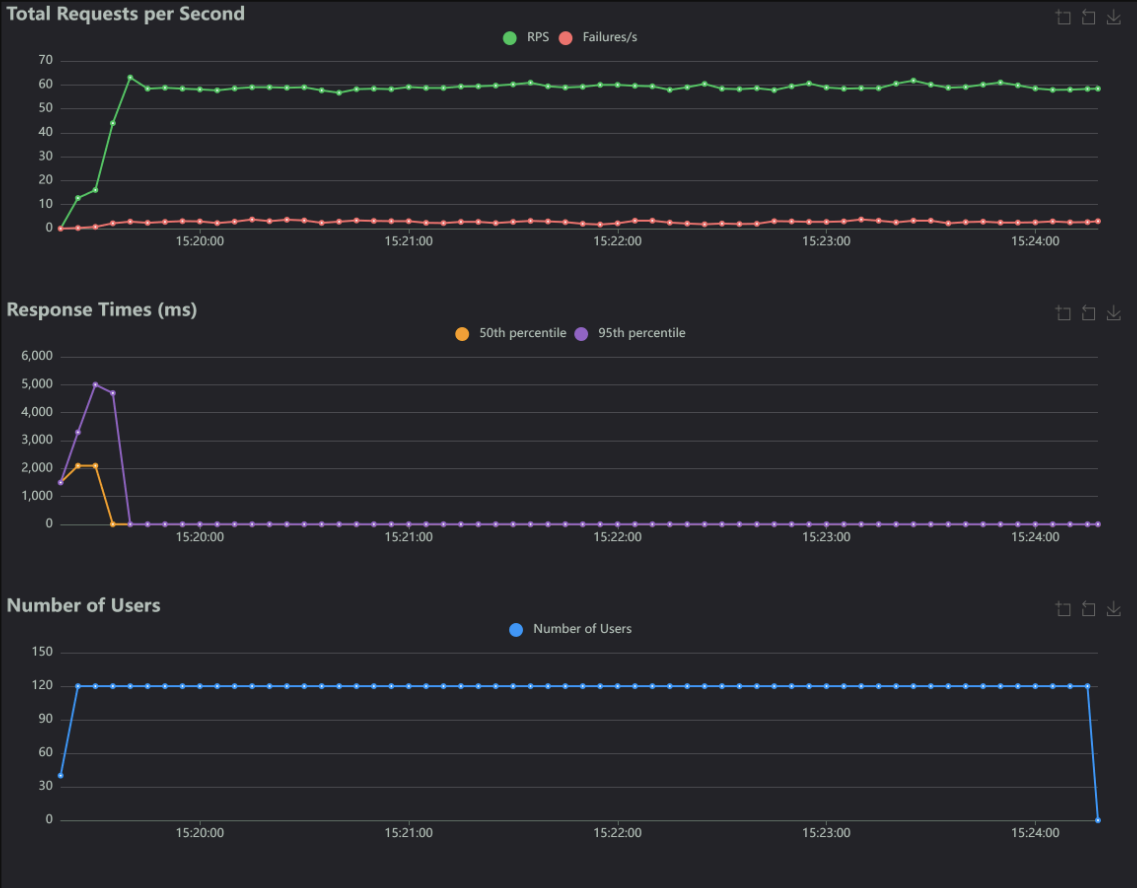
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	Audit Stats	5	5	6	6	6	7	8	8
GET	Dashboard Charts	8	8	8	9	9	11	3200	5400
GET	Dashboard KPIs	6	6	6	7	8	8	180	5300
GET	Get My Appointments	6	6	6	7	7	8	87	5600
GET	Get Patient Info	6	6	6	6	7	8	1400	4100
GET	Get User by ID	4	4	4	5	5	6	290	1400
GET	Health Check	2	2	2	3	3	4	11	2500
GET	Health Database	4	4	4	4	5	5	10	4100
GET	List Appointments	6	6	6	7	8	8	13	4700
GET	List Audit Logs	6	6	6	7	8	9	3300	3300
GET	List Consultations	6	6	6	7	7	8	19	4700
GET	List Doctors	6	7	7	7	9	10	12	21
GET	List My Appointments	6	6	6	7	8	9	1200	4700
GET	List Patients	17	18	19	20	21	23	1500	4100
GET	List Users	6	6	6	7	8	9	10	99
POST	[1] PRIMERO - Login - Generar Token	3300	3600	4600	4700	5000	5400	5400	5700
GET	[2] Get Current User	4	4	5	5	5	6	33	4100
	Aggregated	5	5	6	6	7	8	2100	5700

Failures Statistics

# Failures	Method	Name	Message
816	GET	Get Patient Info	HTTPError('404 Client Error: Not Found for url: Get Patient Info')
2	POST	[1] PRIMERO - Login - Generar Token	CatchResponseError('✗ Login falló: código 500')

Charts



5- RAMPUP

Locust Test Report

During: 22/11/2025, 15:25:14 - 22/11/2025, 15:27:14 (2 minutes)
Target Host: http://localhost:5000
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	Audit Stats	5	0	5.66	5	6	210	0.04	0
GET	Dashboard Charts	13	0	7.63	6	11	337	0.11	0
GET	Dashboard KPIs	14	0	5.62	4	7	56	0.12	0
GET	Get My Appointments	275	0	6.52	4	173	58	2.29	0
GET	Get Patient Info	50	47	5.91	4	9	58.7	0.42	0.39
GET	Get User by ID	13	0	4.01	4	5	108.85	0.11	0
GET	Health Check	221	0	4.16	1	257	228.89	1.84	0
GET	Health Database	97	0	4.87	2	85	284	0.81	0
GET	List Appointments	8	0	6.76	6	8	58	0.07	0
GET	List Audit Logs	6	0	5.79	5	7	792	0.05	0
GET	List Consultations	74	0	7.9	4	161	67.5	0.62	0
GET	List Doctors	7	0	6.15	5	7	2211	0.06	0
GET	List My Appointments	79	0	8.78	5	216	58	0.66	0
GET	List Patients	5	0	18.46	15	27	2440	0.04	0
GET	List Users	21	0	31.35	5	340	1469.62	0.18	0
POST	[1] PRIMERO - Login - Generar Token	20	0	296.85	89	452	287.4	0.17	0
GET	[2] Get Current User	292	0	5.15	3	160	88.71	2.44	0
Aggregated		1200	47	11.12	1	452	174.69	10.01	0.39



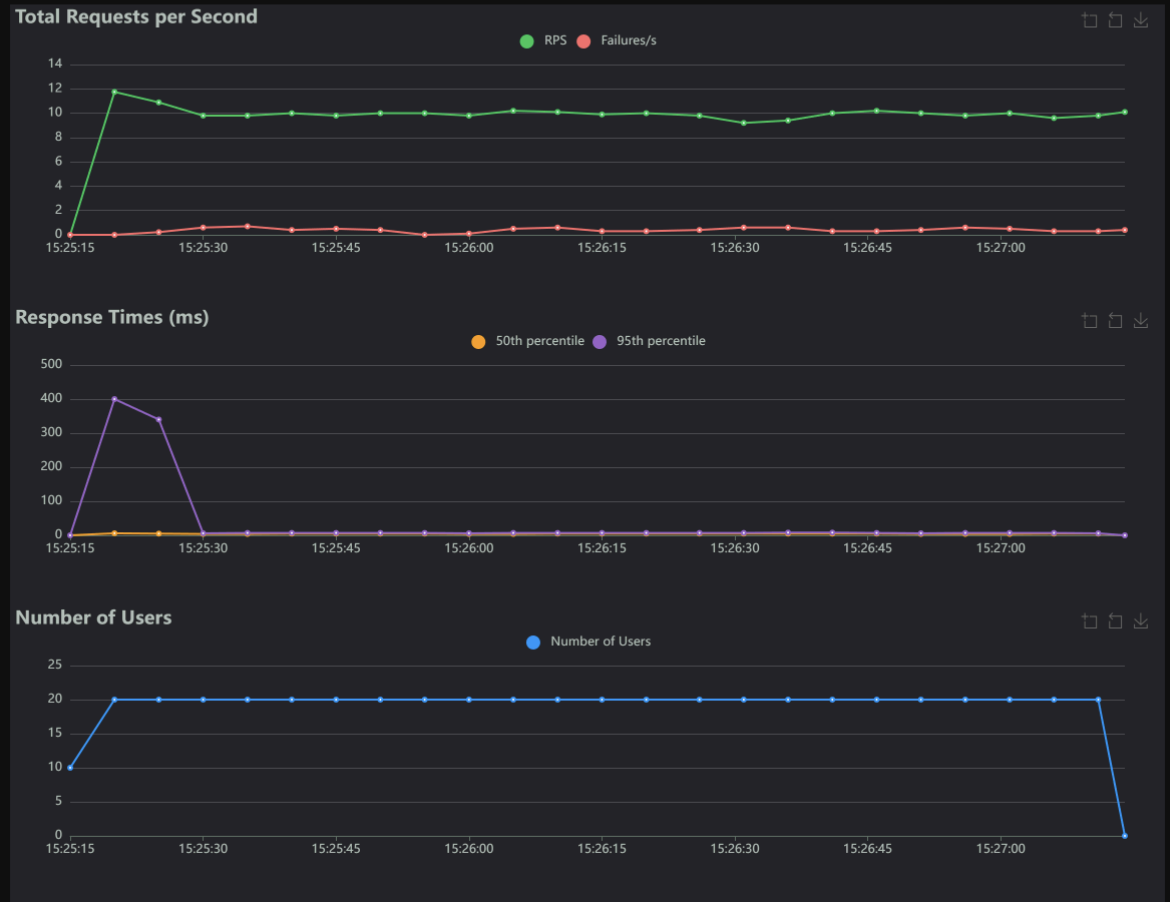
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	Audit Stats	6	6	6	6	6	6	6	6
GET	Dashboard Charts	7	7	8	8	9	11	11	11
GET	Dashboard KPIs	6	6	6	6	6	7	7	7
GET	Get My Appointments	6	6	6	6	7	7	7	170
GET	Get Patient Info	6	6	6	6	7	7	9	9
GET	Get User by ID	4	4	4	4	5	5	5	5
GET	Health Check	2	2	2	2	3	3	3	260
GET	Health Database	4	4	4	4	4	5	85	85
GET	List Appointments	7	7	7	7	8	8	8	8
GET	List Audit Logs	6	6	6	6	7	7	7	7
GET	List Consultations	6	6	6	6	7	8	160	160
GET	List Doctors	6	6	6	7	7	7	7	7
GET	List My Appointments	6	6	6	6	7	7	220	220
GET	List Patients	16	18	18	27	27	27	27	27
GET	List Users	6	6	6	7	8	210	340	340
POST	[1] PRIMERO - Login - Generar Token	370	400	400	410	440	450	450	450
GET	[2] Get Current User	4	4	4	4	5	5	44	160
	Aggregated	5	5	6	6	7	7	260	450

Failures Statistics

# Failures	Method	Name	Message
47	GET	Get Patient Info	HTTPError(404 Client Error: Not Found for url: Get Patient Info')

Charts



6- SPIKE

Locust Test Report

50 %



Re

During: 22/11/2025, 15:33:36 - 22/11/2025, 15:34:36 (1 minute)
Target Host: http://localhost:5000
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	Audit Stats	21	0	13	5	140	210	0.35	0
GET	Dashboard Charts	51	1	72.36	7	3212	331.39	0.85	0.02
GET	Dashboard KPIs	64	0	240.37	4	5300	56	1.07	0
GET	Get My Appointments	214	4	279.47	4	5260	57.87	3.57	0.07
GET	Get Patient Info	238	219	126.5	4	4061	72.19	3.97	3.65
GET	Get User by ID	45	0	10.62	3	123	110.44	0.75	0
GET	Health Check	1302	5	76.95	1	2413	228.02	21.71	0.08
GET	Health Database	639	0	140.64	2	4700	284.12	10.66	0
GET	List Appointments	43	0	127.58	5	5037	58	0.72	0
GET	List Audit Logs	20	0	13.94	5	145	792	0.33	0
GET	List Consultations	259	3	185.55	4	6622	85.06	4.32	0.05
GET	List Doctors	43	1	155.19	5	4366	2160.77	0.72	0.02
GET	List My Appointments	358	8	181.57	4	5211	57.84	5.97	0.13
GET	List Patients	32	0	20.84	13	30	2440	0.53	0
GET	List Users	78	0	170.75	4	5163	1342.72	1.3	0
POST	[1] PRIMERO - Login - Generar Token	300	199	5060.27	265	7861	131.1	5	3.32
GET	[2] Get Current User	578	1	111.33	3	4952	88.63	9.64	0.02
Aggregated		4285	441	471.81	1	7861	225.26	71.45	7.35



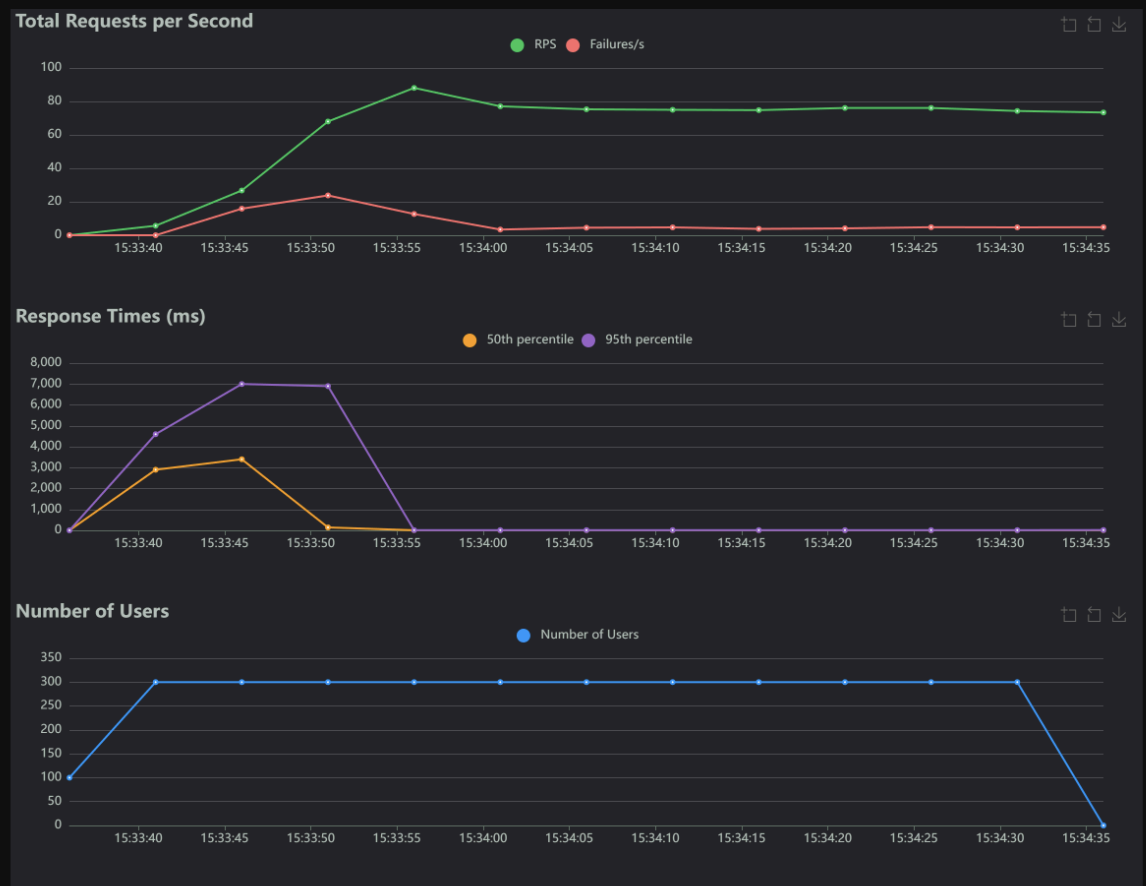
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	Audit Stats	7	7	7	9	9	9	140	140
GET	Dashboard Charts	9	9	10	11	13	15	3200	3200
GET	Dashboard KPIs	7	8	8	9	77	2000	5300	5300
GET	Get My Appointments	7	7	8	9	11	3200	5300	5300
GET	Get Patient Info	7	7	7	8	9	11	3500	4100
GET	Get User by ID	5	5	6	7	10	14	120	120
GET	Health Check	3	4	4	5	8	360	1800	2400
GET	Health Database	5	5	6	7	9	1300	3400	4700
GET	List Appointments	7	8	8	9	10	11	5000	5000
GET	List Audit Logs	7	7	8	8	13	140	140	140
GET	List Consultations	7	7	8	8	10	130	5100	6600
GET	List Doctors	8	8	9	9	11	13	4400	4400
GET	List My Appointments	7	7	8	9	11	1900	5200	5200
GET	List Patients	21	22	22	23	24	29	30	30
GET	List Users	7	8	9	9	12	610	5200	5200
POST	[1] PRIMERO - Login - Generar Token	5300	6000	6000	6000	7000	7800	7800	7900
GET	[2] Get Current User	5	5	6	6	8	120	3500	5000
	Aggregated	6	6	7	9	1800	5100	6600	7900

Failures Statistics

# Failures	Method	Name	Message
1	GET	Dashboard Charts	HTTPError('500 Server Error: Internal Server Error for url: Dashboard Charts')
4	GET	Get My Appointments	HTTPError('500 Server Error: Internal Server Error for url: Get My Appointments')
219	GET	Get Patient Info	HTTPError('404 Client Error: Not Found for url: Get Patient Info')
5	GET	Health Check	ConnectionResetError(10054, 'Se ha forzado la interrupción de una conexión existente por el host remoto', None, 10054, None)
3	GET	List Consultations	HTTPError('500 Server Error: Internal Server Error for url: List Consultations')
1	GET	List Doctors	HTTPError('500 Server Error: Internal Server Error for url: List Doctors')
8	GET	List My Appointments	HTTPError('500 Server Error: Internal Server Error for url: List My Appointments')
199	POST	[1] PRIMERO - Login - Generar Token	CatchResponseError(❌ Login falló: código 500')
1	GET	[2] Get Current User	HTTPError('500 Server Error: Internal Server Error for url: [2] Get Current User')

Charts



7- SOAK

Locust Test Report

During: 22/11/2025, 15:35:18 - 22/11/2025, 16:05:18 (30 minutes)

Target Host: http://localhost:5000

Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	Audit Stats	117	0	6.25	3	12	210	0.07	0
GET	Dashboard Charts	347	0	9.4	6	43	337	0.19	0
GET	Dashboard KPIs	449	0	6.6	4	12	56	0.25	0
GET	Get My Appointments	8445	0	7.12	4	343	58	4.69	0
GET	Get Patient Info	1858	1694	7.23	4	297	79.85	1.03	0.94
GET	Get User by ID	367	0	4.91	3	9	110.49	0.2	0
GET	Health Check	5781	0	2.74	1	285	228.89	3.21	0
GET	Health Database	2969	0	4.41	2	326	284	1.65	0
GET	List Appointments	238	0	7.34	5	22	58	0.13	0
GET	List Audit Logs	239	0	6.86	3	24	792	0.13	0
GET	List Consultations	2344	0	6.94	4	259	70	1.3	0
GET	List Doctors	235	0	7.35	5	12	2211	0.13	0
GET	List My Appointments	2768	0	7.07	4	264	58	1.54	0
GET	List Patients	258	0	20.3	12	45	2440	0.14	0
GET	List Users	589	0	7.3	4	152	937.39	0.33	0
POST	[1] PRIMERO - Login - Generar Token	40	0	291.78	85	413	287.25	0.02	0
GET	[2] Get Current User	8769	0	5.13	2	306	88.68	4.87	0
Aggregated		35813	1694	6.1	1	413	168.37	19.9	0.94



Response Time Statistics

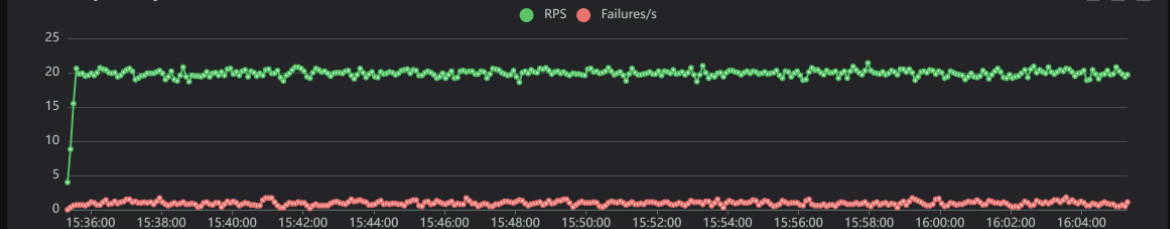
Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	Audit Stats	6	6	7	7	8	8	9	12
GET	Dashboard Charts	9	9	10	10	11	12	17	43
GET	Dashboard KPIs	6	7	7	7	8	9	11	12
GET	Get My Appointments	7	7	7	8	8	9	12	340
GET	Get Patient Info	6	7	7	7	8	9	13	300
GET	Get User by ID	5	5	5	5	6	6	8	9
GET	Health Check	2	3	3	3	3	4	5	280
GET	Health Database	4	4	4	5	5	6	7	330
GET	List Appointments	7	7	8	8	9	10	12	22
GET	List Audit Logs	7	7	7	8	8	9	14	24
GET	List Consultations	7	7	7	8	8	9	11	260
GET	List Doctors	7	7	8	8	9	9	12	12
GET	List My Appointments	7	7	7	8	8	9	12	260
GET	List Patients	20	21	22	23	25	29	39	45
GET	List Users	7	7	8	8	8	9	11	150
POST	[1] PRIMERO - Login - Generar Token	350	360	380	380	410	410	410	410
GET	[2] Get Current User	5	5	5	5	6	6	8	310
	Aggregated	6	6	7	7	8	9	17	410

Failures Statistics

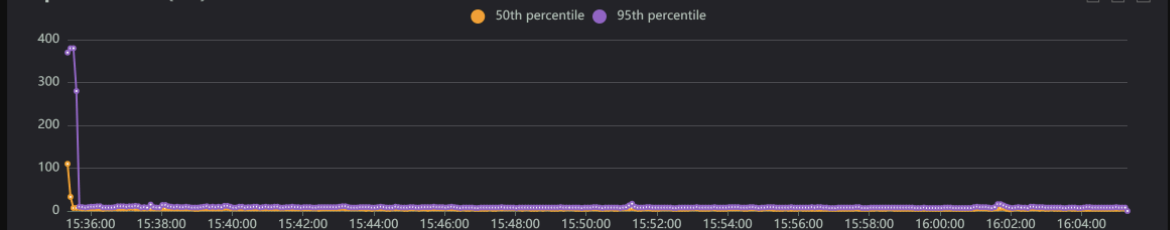
# Failures	Method	Name	Message
1694	GET	Get Patient Info	HTTPError('404 Client Error: Not Found for url: Get Patient Info')

Charts

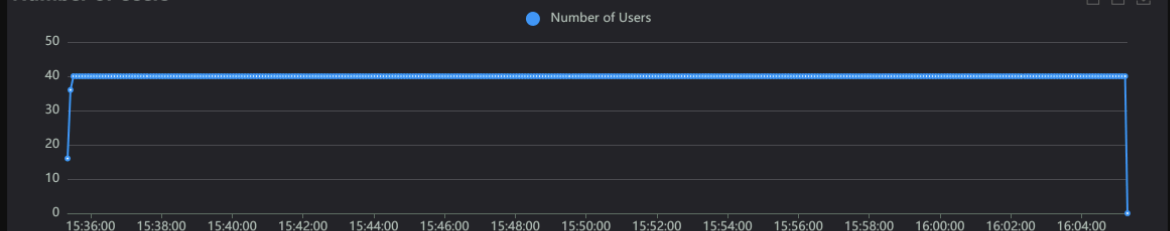
Total Requests per Second



Response Times (ms)



Number of Users



8- LIMIT

Locust Test Report

During: 22/11/2025, 16:17:03 - 22/11/2025, 16:17:33 (30 seconds)
Target Host: http://localhost:5000
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	Audit Stats	7	0	125.42	6	829	210	0.23	0
GET	Dashboard Charts	23	3	990.15	6	7036	299.7	0.77	0.1
GET	Dashboard KPIs	23	1	165.89	5	2495	55.78	0.77	0.03
GET	Get My Appointments	383	31	416.85	5	5917	57.43	12.77	1.03
GET	Get Patient Info	42	37	818.61	4	4043	97.6	1.4	1.23
GET	Get User by ID	22	2	325.93	4	2502	104.09	0.73	0.07
GET	Health Check	463	1	179.82	1	1910	229.4	15.44	0.03
GET	Health Database	242	17	424.31	3	3904	272.94	8.07	0.57
GET	List Appointments	14	2	722.89	7	4021	57	0.47	0.07
GET	List Audit Logs	20	1	674.54	5	4023	754.95	0.67	0.03
GET	List Consultations	69	5	507.49	5	4226	67.68	2.3	0.17
GET	List Doctors	12	1	318.83	7	2492	2031	0.4	0.03
GET	List My Appointments	66	4	901.73	5	6262	57.58	2.2	0.13
GET	List Patients	10	1	952.68	20	4958	2201.1	0.33	0.03
GET	List Users	26	3	752.06	5	5913	907	0.87	0.1
POST	[1] PRIMERO - Login - Generar Token	300	171	7310.14	3908	11360	152.92	10	5.7
GET	[2] Get Current User	252	13	386.7	3	4436	86.77	8.4	0.43
Aggregated		1974	293	1446.88	1	11360	188.64	65.81	9.77



Response Time Statistics

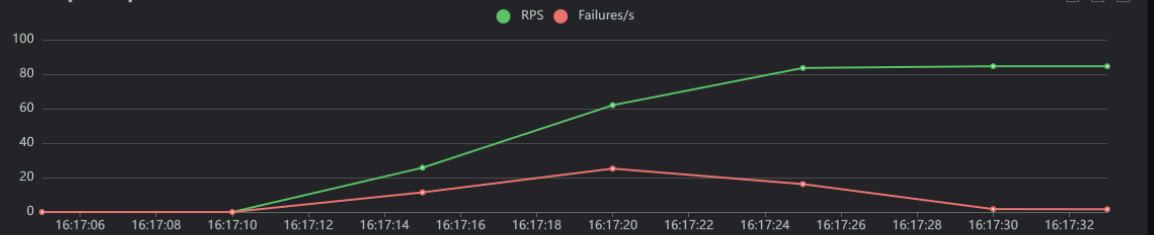
Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	Audit Stats	8	10	10	11	830	830	830	830
GET	Dashboard Charts	12	14	19	3000	4200	5100	7000	7000
GET	Dashboard KPIs	8	9	10	11	12	1100	2500	2500
GET	Get My Appointments	9	10	11	14	2200	2700	5700	5900
GET	Get Patient Info	10	12	930	2000	3500	4000	4000	4000
GET	Get User by ID	6	6	7	8	1100	2500	2500	2500
GET	Health Check	4	5	8	140	860	1200	1600	1900
GET	Health Database	6	7	9	330	2500	2700	3800	3900
GET	List Appointments	11	11	11	2500	2700	4000	4000	4000
GET	List Audit Logs	8	8	9	2600	4000	4000	4000	4000
GET	List Consultations	9	10	11	14	4000	4000	4200	4200
GET	List Doctors	9	10	11	14	1200	2500	2500	2500
GET	List My Appointments	9	10	15	2600	4000	4200	6300	6300
GET	List Patients	28	30	42	4300	5000	5000	5000	5000
GET	List Users	9	10	11	19	3200	5800	5900	5900
POST	[1] PRIMERO - Login - Generar Token	7000	8600	8700	8700	9600	11000	11000	11000
GET	[2] Get Current User	6	6	7	10	2000	3200	4400	4400
	Aggregated	9	11	280	2700	7000	8600	11000	11000

Failures Statistics

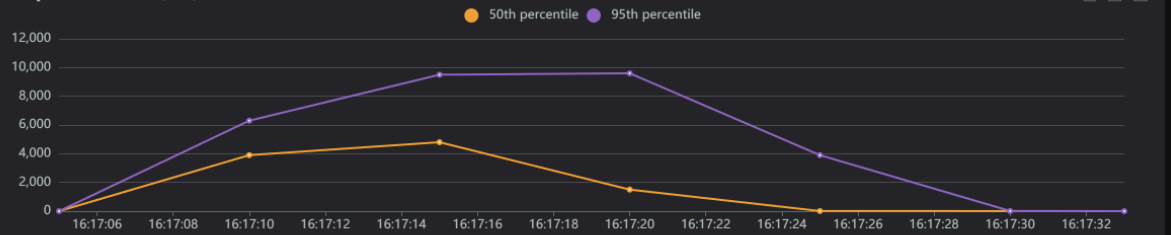
# Failures	Method	Name	Message
3	GET	Dashboard Charts	HTTPError('500 Server Error: Internal Server Error for url: Dashboard Charts')
1	GET	Dashboard KPIs	HTTPError('500 Server Error: Internal Server Error for url: Dashboard KPIs')
31	GET	Get My Appointments	HTTPError('500 Server Error: Internal Server Error for url: Get My Appointments')
5	GET	Get Patient Info	HTTPError('500 Server Error: Internal Server Error for url: Get Patient Info')
32	GET	Get Patient Info	HTTPError('404 Client Error: Not Found for url: Get Patient Info')
2	GET	Get User by ID	HTTPError('500 Server Error: Internal Server Error for url: Get User by ID')
1	GET	Health Check	ConnectionResetError(10054, 'Se ha forzado la interrupción de una conexión existente por el host remoto', None, 10054, None)
3	GET	Health Database	ConnectionResetError(10054, 'Se ha forzado la interrupción de una conexión existente por el host remoto', None, 10054, None)
14	GET	Health Database	HTTPError('503 Server Error: Service Unavailable for url: Health Database')
2	GET	List Appointments	HTTPError('500 Server Error: Internal Server Error for url: List Appointments')
1	GET	List Audit Logs	HTTPError('500 Server Error: Internal Server Error for url: List Audit Logs')
5	GET	List Consultations	HTTPError('500 Server Error: Internal Server Error for url: List Consultations')
1	GET	List Doctors	HTTPError('500 Server Error: Internal Server Error for url: List Doctors')
4	GET	List My Appointments	HTTPError('500 Server Error: Internal Server Error for url: List My Appointments')
1	GET	List Patients	HTTPError('500 Server Error: Internal Server Error for url: List Patients')
3	GET	List Users	HTTPError('500 Server Error: Internal Server Error for url: List Users')
171	POST	[1] PRIMERO - Login - Generar Token	CatchResponseError('✖ Login falló: código 500')
13	GET	[2] Get Current User	HTTPError('500 Server Error: Internal Server Error for url: [2] Get Current User')

Charts

Total Requests per Second



Response Times (ms)



Number of Users

